

实验指导书（二）

- 零、下载实验工程
- 一、OVS查看交换机流表
- 二、Wireshark对控制器抓包
- 三、如何运行OS-Ken APP
- 四、OS-Ken编程示例：简单交换机
 - 1. 代码
 - 2. 运行
- 五、实验任务
 - 1. 实验任务一：自学习交换机
 - 问题说明
 - 代码框架
 - 结果示例
 - 2. 实验任务二：处理环路广播
 - 代码框架
 - 结果示例
 - 3. 附加题
 - 4. 拓展资料

零、下载实验工程

1. 下载UV Python包管理器

我们使用UV来方便大家统一版本。如果你使用了我们的镜像，或者使用的是archlinux：

```
# （可选）更新软件列表
sudo pacman -Syy

# 安装实验所使用的Python包管理器
sudo pacman -S uv
```

Fedora / RHEL

```
sudo dnf install uv
```

其他发行版

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

2. 获取实验Git仓库

```
git clone https://github.com/XJTU-NetVerify/sdn-lab2.git
cd sdn-lab2
```

3. 安装依赖和初始化

```
uv sync
```

4. 运行OS-Ken App

```
uv run osken-manager simple_switch.py
```

如果你不希望再反复输入 `uv run`，你可以使用Python虚拟环境（virtualenv, `venv`）；此后你就可以直接运行：

```
# 创建虚拟环境：
uv venv
uv sync

# 创建完虚拟环境后，每次进入新的shell你需要运行：
source .venv/bin/activate

# 此时和Conda一样，Shell prompt前面会有`(sdn-lab2)`提示，即说明你在`venv`虚拟环境中。
# 你可以直接运行osken-manager。
osken-manager simple_switch.py

# 如果要退出`venv`虚拟环境：
deactivate
```

一、OVS查看交换机流表

- 查看交换机 `s1` 的流表

```
sudo ovs-ofctl dump-flows s1
```

```
sdn@ubuntu:~/Desktop$ sudo ovs-ofctl dump-flows s1
cookie=0x0, duration=662.646s, table=0, n_packets=24, n_bytes=1832, priority=0
actions=CONTROLLER:128
```

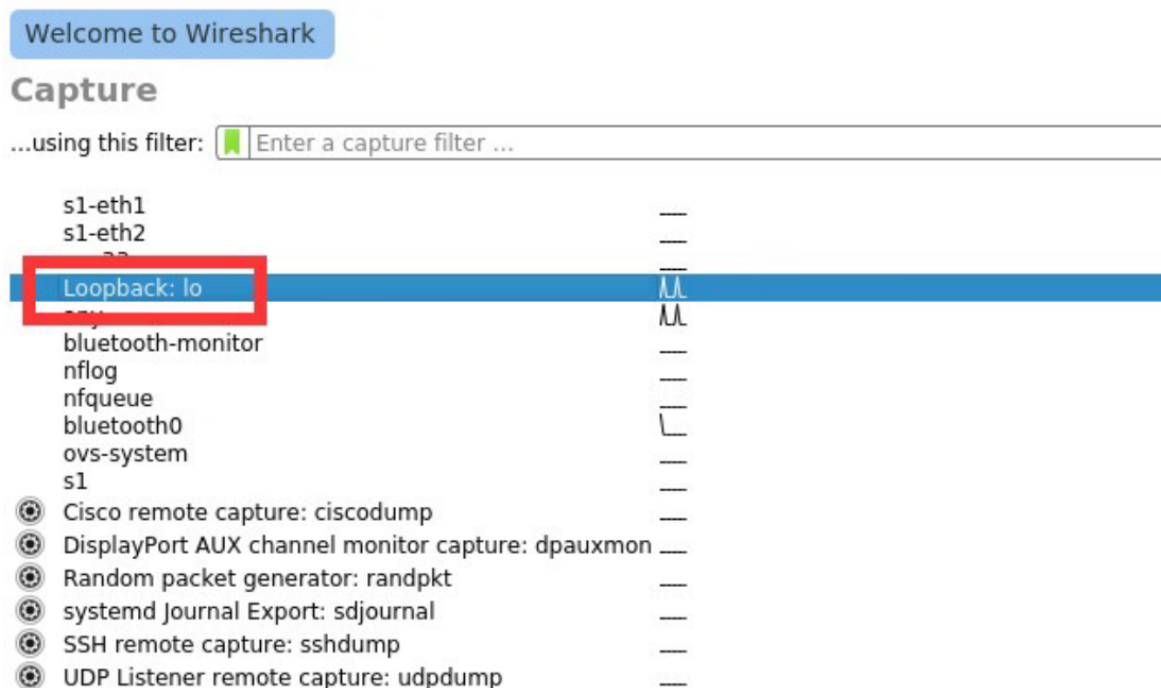
- 只查看 `OpenFlow1.3` 的流表项

```
sudo ovs-ofctl -O OpenFlow13 dump-flows s1
```

```
sdn@ubuntu:~/Desktop$ sudo ovs-ofctl -O OpenFlow13 dump-flows s1
cookie=0x0, duration=717.490s, table=0, n_packets=24, n_bytes=1832, priority=0
actions=CONTROLLER:128
```

二、Wireshark对控制器抓包

启动 WireShark，监听 Loopback:lo 端口。



三、如何运行OS-Ken APP

Ryu现已不再维护；因此，我们在实验中选用从Ryu fork的，目前正在积极维护中的**OS-Ken**。其大部分内容和Ryu通用。

以运行 OS-Ken 提供的查看拓扑的 APP 为例：

- mininet 创建拓扑。

```
sudo mn --custom topo_2sw_2host.py --topo mytopo --controller remote
#remote表示不用`Mininet`自带的控制器，尝试使用`OS-Ken`等远端控制器。
```

- 运行OS-Ken App。

```
uv run osken-manager simple_switch.py
```

四、OS-Ken编程示例：简单交换机

1. 代码

```
from os_ken.base import app_manager
from os_ken.controller import ofp_event
from os_ken.controller.handler import MAIN_DISPATCHER, CONFIG_DISPATCHER
```

```

from os_ken.controller.handler import set_ev_cls
from os_ken.ofproto import ofproto_v1_3

class L2Switch(app_manager.OSKenApp):
    OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]

    def __init__(self, *args, **kwargs):
        super(L2Switch, self).__init__(*args, **kwargs)

    def add_flow(self, datapath, priority, match, actions):
        dp = datapath
        ofp = dp.ofproto
        parser = dp.ofproto_parser

        inst = [parser.OFPInstructionActions(ofp.OFPIT_APPLY_ACTIONS,
actions)]
        mod = parser.OFPFlowMod(datapath=dp, priority=priority, match=match,
instructions=inst)
        dp.send_msg(mod)

    # add default flow table which sends packets to the controller
    @set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
    def switch_features_handler(self, ev):
        msg = ev.msg
        dp = msg.datapath
        ofp = dp.ofproto
        parser = dp.ofproto_parser

        match = parser.OFPMatch()
        actions = [parser.OFPActionOutput(ofp.OFPP_CONTROLLER,
ofp.OFPCML_NO_BUFFER)]
        self.add_flow(dp, 0, match, actions)

    # handle packet_in message
    @set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
    def packet_in_handler(self, ev):
        msg = ev.msg
        dp = msg.datapath
        ofp = dp.ofproto
        parser = dp.ofproto_parser

        actions = [parser.OFPActionOutput(ofp.OFPP_FLOOD)]
        out = parser.OFPPacketOut(
            datapath=dp, buffer_id=msg.buffer_id,
in_port=msg.match['in_port'], actions=actions, data=msg.data)
        dp.send_msg(out)

```

2. 运行

- mininet 创建树状拓扑

```
sudo mn --controller remote --mac --topo=tree,2,2
```

- 将上述代码保存为 simple_switch.py, 启动控制器

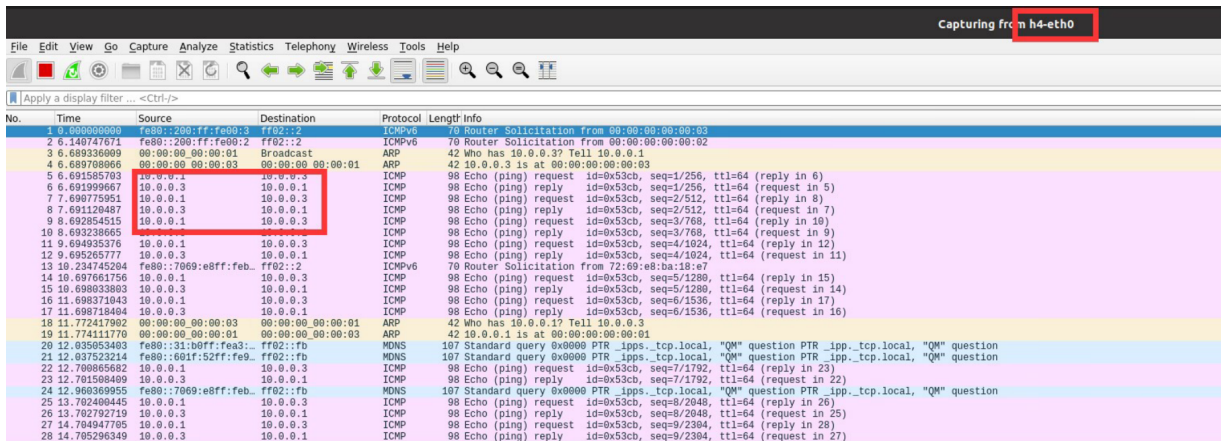
```
uv run osken-manager simple_switch.py
```

- 在 mininet 的 CLI 中启动 wireshark 抓取端口 h4-eth0

```
mininet> h4 wireshark &
```

- 在 mininet 中 h1 ping h3, 查看 wireshark 中关于 h4 的抓包情况

```
mininet> h1 ping h3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=5.06 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=2.66 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=2.73 ms
64 bytes from 10.0.0.3: icmp_seq=4 ttl=64 time=2.77 ms
64 bytes from 10.0.0.3: icmp_seq=5 ttl=64 time=2.69 ms
64 bytes from 10.0.0.3: icmp_seq=6 ttl=64 time=2.49 ms
```

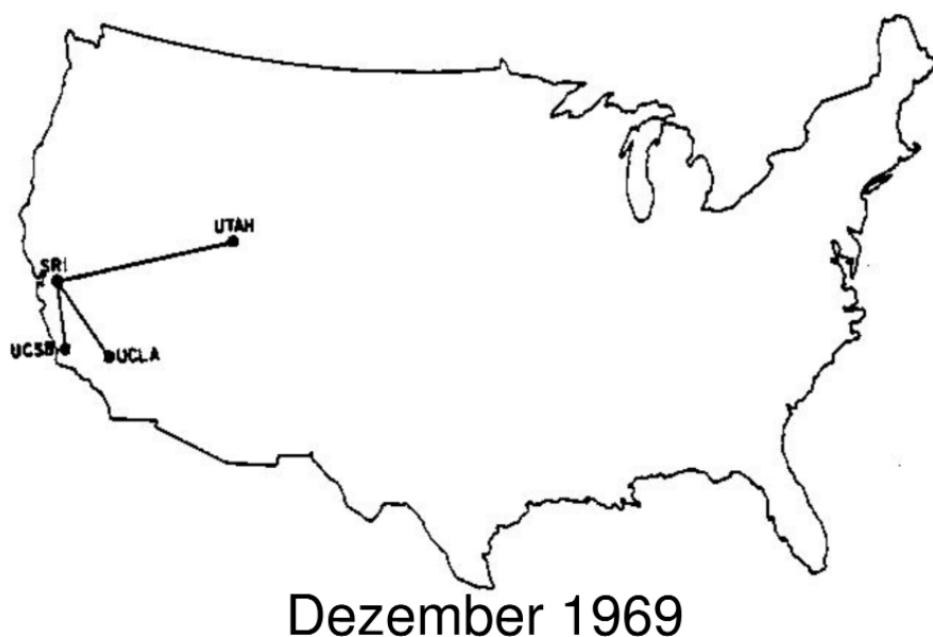


No.	Time	Source	Destination	Protocol	Length	Info
1	0.0000000	00:00:00:00:00:00	ff02::2	ICMPv6	70	Router Solicitation from 00:00:00:00:00:00
2	6.140747071	fe80::200:ff:fe00:2	ff02::2	ICMPv6	42	Who has 10.0.0.3? Tell 10.0.0.1
3	6.689336009	00:00:00:00:00:01	Broadcast	ARP	42	Who has 10.0.0.3? Tell 10.0.0.1
4	6.689708066	00:00:00:00:00:03	00:00:00:00:00:01	ARP	42	Who has 10.0.0.3? Tell 10.0.0.1
5	6.691585703	10.0.0.1	10.0.0.3	ICMP	98	Echo (ping) request id=0x53cb, seq=1/256, ttl=64 (reply in 6)
6	6.691999667	10.0.0.3	10.0.0.1	ICMP	98	Echo (ping) reply id=0x53cb, seq=1/256, ttl=64 (request in 5)
7	7.690775951	10.0.0.1	10.0.0.3	ICMP	98	Echo (ping) request id=0x53cb, seq=2/512, ttl=64 (reply in 8)
8	7.691120487	10.0.0.3	10.0.0.1	ICMP	98	Echo (ping) reply id=0x53cb, seq=2/512, ttl=64 (request in 7)
9	8.692854515	10.0.0.1	10.0.0.3	ICMP	98	Echo (ping) request id=0x53cb, seq=3/768, ttl=64 (reply in 10)
10	8.693238665	10.0.0.3	10.0.0.1	ICMP	98	Echo (ping) reply id=0x53cb, seq=3/768, ttl=64 (request in 9)
11	9.694095376	10.0.0.1	10.0.0.3	ICMP	98	Echo (ping) request id=0x53cb, seq=4/1024, ttl=64 (reply in 12)
12	9.695265777	10.0.0.3	10.0.0.1	ICMP	98	Echo (ping) reply id=0x53cb, seq=4/1024, ttl=64 (request in 11)
13	10.234745204	fe80::7069:e8ff:feb...	ff02::2	ICMPv6	70	Router Solicitation from 72:69:e8:ba:18:e7
14	10.697661756	10.0.0.1	10.0.0.3	ICMP	98	Echo (ping) request id=0x53cb, seq=5/1280, ttl=64 (reply in 15)
15	10.698033803	10.0.0.3	10.0.0.1	ICMP	98	Echo (ping) reply id=0x53cb, seq=5/1280, ttl=64 (request in 14)
16	11.698371043	10.0.0.1	10.0.0.3	ICMP	98	Echo (ping) request id=0x53cb, seq=6/1536, ttl=64 (reply in 17)
17	11.698719404	10.0.0.3	10.0.0.1	ICMP	98	Echo (ping) reply id=0x53cb, seq=6/1536, ttl=64 (request in 16)
18	11.724117902	00:00:00:00:00:03	00:00:00:00:00:01	ARP	42	Who has 10.0.0.1? Tell 10.0.0.3
19	11.72411770	00:00:00:00:00:01	00:00:00:00:00:03	ARP	42	Who has 10.0.0.1? Tell 10.0.0.3
20	12.035653403	fe80::31:b0ff:fea3...	ff02::fb	MDNS	107	Standard query 0x0000 PTR _ipps._tcp.local, "QM" question PTR _ipps._tcp.local, "QM" question
21	12.037523214	fe80::601f:52ff:fea...	ff02::fb	MDNS	107	Standard query 0x0000 PTR _ipps._tcp.local, "QM" question PTR _ipps._tcp.local, "QM" question
22	12.700565682	10.0.0.1	10.0.0.3	ICMP	98	Echo (ping) request id=0x53cb, seq=7/1792, ttl=64 (reply in 23)
23	12.701508409	10.0.0.3	10.0.0.1	ICMP	98	Echo (ping) reply id=0x53cb, seq=7/1792, ttl=64 (request in 22)
24	12.960369955	fe80::7069:e8ff:feb...	ff02::fb	MDNS	107	Standard query 0x0000 PTR _ipps._tcp.local, "QM" question PTR _ipps._tcp.local, "QM" question
25	13.702400445	10.0.0.1	10.0.0.3	ICMP	98	Echo (ping) request id=0x53cb, seq=8/2048, ttl=64 (reply in 26)
26	13.702792719	10.0.0.3	10.0.0.1	ICMP	98	Echo (ping) reply id=0x53cb, seq=8/2048, ttl=64 (request in 25)
27	14.704947705	10.0.0.1	10.0.0.3	ICMP	98	Echo (ping) request id=0x53cb, seq=9/2304, ttl=64 (reply in 28)
28	14.705296349	10.0.0.3	10.0.0.1	ICMP	98	Echo (ping) reply id=0x53cb, seq=9/2304, ttl=64 (request in 27)

抓包结果显示该交换机存在验证缺陷: packet_in_handler 函数会将数据包洪泛到交换机的所有端口, 故 h1 和 h3 通讯时, h4 也会收到所有的包。

五、实验任务

1. 实验任务一：自学习交换机



1969年的 ARPANET 非常简单，仅由四个结点组成。假设每个结点都对应一个交换机，每个交换机都具有一个直连主机，你的任务是实现不同主机之间的正常通信。

前文给出的简单交换机洪泛数据包，虽然能初步实现主机间的通信，但会带来不必要的带宽消耗，并且会使通信内容泄露给第三者。因此，请在简单交换机的基础上实现二层自学习交换机，避免数据包的洪泛。

问题说明

- SDN 自学习交换机的工作流程可以参考：
 1. 控制器为每个交换机维护一个 `mac-port` 映射表。
 1. 控制器收到 `packet_in` 消息后，解析其中携带的数据包。
 1. 控制器学习 `src_mac - in_port` 映射。
 1. 控制器查询 `dst_mac`，如果未学习，则洪泛数据包；如果已学习，则向指定端口转发数据包(`packet_out`)，并向交换机下发流表项(`flow_mod`)，指导交换机转发同类型的数据包。
- 网络拓扑为 `topo_1969_1.py`，启动方式：

```
sudo ./topo_1969_1.py
```

- 可以不考虑交换机对数据包的缓存(`no_buffer`)。

代码框架

以下给出代码框架，只需补充关键的若干行实现即可：

```
from os_ken.base import app_manager
from os_ken.controller import ofp_event
from os_ken.controller.handler import MAIN_DISPATCHER, CONFIG_DISPATCHER
```

```

from os_ken.controller.handler import set_ev_cls
from os_ken.ofproto import ofproto_v1_3
from os_ken.lib.packet import packet
from os_ken.lib.packet import ethernet

class Switch(app_manager.OSKenApp):

    OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]

    def __init__(self, *args, **kwargs):
        super(Switch, self).__init__(*args, **kwargs)
        # maybe you need a global data structure to save the mapping

    def add_flow(self, datapath, priority, match,
actions,idle_timeout=0,hard_timeout=0):
        dp = datapath
        ofp = dp.ofproto
        parser = dp.ofproto_parser
        inst = [parser.OFPInstructionActions(ofp.OFPIT_APPLY_ACTIONS,
actions)]
        mod = parser.OFPFlowMod(datapath=dp, priority=priority,
                                idle_timeout=idle_timeout,
                                hard_timeout=hard_timeout,
                                match=match,instructions=inst)
        dp.send_msg(mod)

    @set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
    def switch_features_handler(self, ev):
        msg = ev.msg
        dp = msg.datapath
        ofp = dp.ofproto
        parser = dp.ofproto_parser
        match = parser.OFPMatch()
        actions =
[parser.OFPActionOutput(ofp.OFPP_CONTROLLER,ofp.OFPCML_NO_BUFFER)]
        self.add_flow(dp, 0, match, actions)

    @set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
    def packet_in_handler(self, ev):
        msg = ev.msg
        dp = msg.datapath
        ofp = dp.ofproto
        parser = dp.ofproto_parser

        # the identity of switch
        dpid = dp.id
        # the port that receive the packet
        in_port = msg.match['in_port']
        pkt = packet.Packet(msg.data)
        eth_pkt = pkt.get_protocol(ethernet.ethernet)

```

```
# get the mac
dst = eth_pkt.dst
src = eth_pkt.src
# we can use the logger to print some useful information
self.logger.info('packet: %s %s %s %s', dpid, src, dst, in_port)

# You need to code here to avoid the direct flooding
# Have fun!
# :)
```

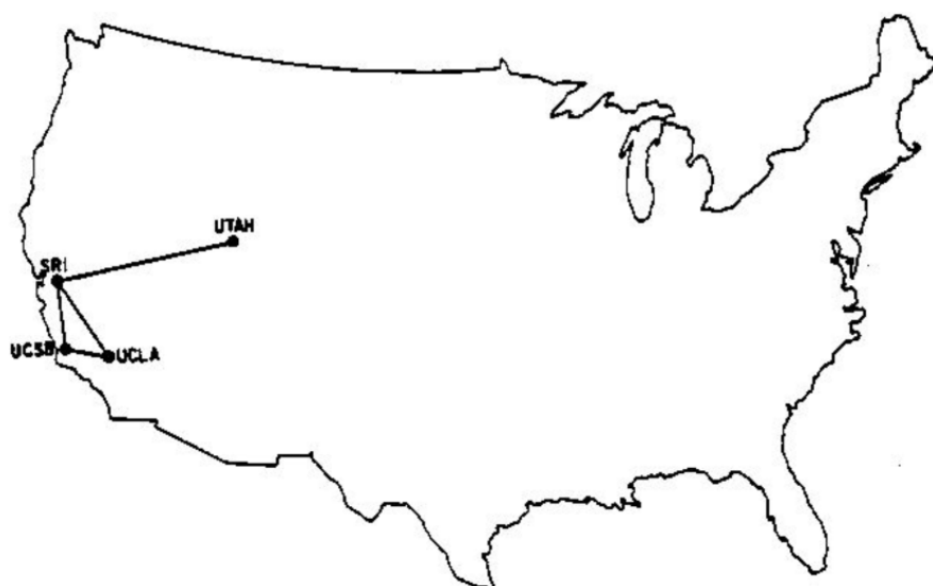
结果示例

UCLA ping UTAH, UCSB 不再收到相关数据包：

```
mininet> UCLA ping UTAH
PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data.
64 bytes from 10.0.0.4: icmp_seq=1 ttl=64 time=269 ms
64 bytes from 10.0.0.4: icmp_seq=2 ttl=64 time=129 ms
64 bytes from 10.0.0.4: icmp_seq=3 ttl=64 time=129 ms
64 bytes from 10.0.0.4: icmp_seq=4 ttl=64 time=127 ms
64 bytes from 10.0.0.4: icmp_seq=5 ttl=64 time=129 ms
64 bytes from 10.0.0.4: icmp_seq=6 ttl=64 time=128 ms
64 bytes from 10.0.0.4: icmp_seq=7 ttl=64 time=128 ms
```

Capturing from UCSB-eth0					
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help					
Apply a display filter ... <Ctrl-/>					
No.	Time	Source	Destination	Protocol	Length Info
1	0.000000000	fe80::f0ed:5eff:fe7...	ff02::2	ICMPv6	70 Router Solicitation from f2:ed:5e:7d:df:b2
2	1.791304424	fe80::b41c:aff:fe27...	ff02::2	ICMPv6	70 Router Solicitation from b6:1c:0a:27:48:2f
3	2.336930409	fe80::f0ed:5eff:fe7...	ff02::fb	MDNS	107 Standard query 0x0000 PTR _ipps._tcp.local, "QM" que
4	10.419047684	76:47:fc:24:d0:bc	Broadcast	ARP	42 Who has 10.0.0.4? tell 10.0.0.2
5	13.092873885	fe80::28b3:fdff:fe4...	ff02::2	ICMPv6	70 Router Solicitation from 2a:b3:fd:46:42:30
6	13.656414129	fe80::ccad:e1ff:fe6...	ff02::2	ICMPv6	70 Router Solicitation from ce:ad:e1:61:ba:c3
7	13.874579064	fe80::8c00:b9ff:fe2...	ff02::2	ICMPv6	70 Router Solicitation from 8e:00:b9:27:4e:e0
8	14.684186968	fe80::e853:63ff:fe2...	ff02::2	ICMPv6	70 Router Solicitation from ea:53:63:2e:3a:d9
9	17.664201656	fe80::f0ed:5eff:fe7...	ff02::2	ICMPv6	70 Router Solicitation from f2:ed:5e:7d:df:b2
10	17.700188001	fe80::ec56:8bff:feb...	ff02::2	ICMPv6	70 Router Solicitation from ee:56:8b:b2:2c:28
11	17.838902677	fe80::e853:63ff:fe2...	ff02::fb	MDNS	107 Standard query 0x0000 PTR _ipps._tcp.local, "QM" que
12	18.235557948	fe80::8c00:b9ff:fe2...	ff02::fb	MDNS	107 Standard query 0x0000 PTR _ipps._tcp.local, "QM" que
13	18.338062389	fe80::f0ed:5eff:fe7...	ff02::fb	MDNS	107 Standard query 0x0000 PTR _ipps._tcp.local, "QM" que
14	18.373943077	fe80::28b3:fdff:fe4...	ff02::fb	MDNS	107 Standard query 0x0000 PTR _ipps._tcp.local, "QM" que
15	19.711766133	fe80::b41c:aff:fe27...	ff02::2	ICMPv6	70 Router Solicitation from b6:1c:0a:27:48:2f
16	19.764846646	fe80::7447:fcff:fe2...	ff02::2	ICMPv6	70 Router Solicitation from 76:47:fc:24:d0:bc
17	42.275722557	fe80::28b3:fdff:fe4...	ff02::2	ICMPv6	70 Router Solicitation from 2a:b3:fd:46:42:30
18	44.376421435	fe80::ccad:e1ff:fe6...	ff02::2	ICMPv6	70 Router Solicitation from ce:ad:e1:61:ba:c3
19	46.387497063	fe80::8c00:b9ff:fe2...	ff02::2	ICMPv6	70 Router Solicitation from 8e:00:b9:27:4e:e0
20	48.470778940	fe80::e853:63ff:fe2...	ff02::2	ICMPv6	70 Router Solicitation from ea:53:63:2e:3a:d9

2. 实验任务二：处理环路广播



Dezember 1969

UCLA 和 UCSB 通信频繁，两者间建立了一条直连链路。在新的拓扑 `topo_1969_2.py` 中运行自学习交换机，UCLA 和 UTAH 之间无法正常通信。分析流表发现，源主机虽然只发了很少的几个数据包，但流表项却匹配了上千次；WireShark 也截取到了数目异常大的相同报文。

```
mininet> dpctl dump-flows
*** s1 -----
cookie=0x0, duration=62.352s, table=0 n_packets=35684, n_bytes=2444122, priority=0 actions=CONTROLLER:65535
*** s2 -----
cookie=0x0, duration=62.355s, table=0 n_packets=33985, n_bytes=2368858, priority=0 actions=CONTROLLER:65535
*** s3 -----
cookie=0x0, duration=62.357s, table=0 n_packets=36993, n_bytes=2497119, priority=0 actions=CONTROLLER:65535
*** s4 -----
cookie=0x0, duration=62.360s, table=0 n_packets=40098, n_bytes=2627809, priority=0 actions=CONTROLLER:65535
```

*UCSB-eth0						
No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	96:78:c6:15:c7:0e	Broadcast	ARP	42	Who has 10.0.0.4? Tell 10.0.0.2
2	0.000360333	96:78:c6:15:c7:0e	Broadcast	ARP	42	Who has 10.0.0.4? Tell 10.0.0.2
3	0.004226763	96:78:c6:15:c7:0e	Broadcast	ARP	42	Who has 10.0.0.4? Tell 10.0.0.2
4	0.006699871	96:78:c6:15:c7:0e	Broadcast	ARP	42	Who has 10.0.0.4? Tell 10.0.0.2
5	0.012695888	96:78:c6:15:c7:0e	Broadcast	ARP	42	Who has 10.0.0.4? Tell 10.0.0.2
6	0.014058793	96:78:c6:15:c7:0e	Broadcast	ARP	42	Who has 10.0.0.4? Tell 10.0.0.2
7	0.016487728	96:78:c6:15:c7:0e	Broadcast	ARP	42	Who has 10.0.0.4? Tell 10.0.0.2
8	0.018252243	96:78:c6:15:c7:0e	Broadcast	ARP	42	Who has 10.0.0.4? Tell 10.0.0.2
9	0.020408389	96:78:c6:15:c7:0e	Broadcast	ARP	42	Who has 10.0.0.4? Tell 10.0.0.2
10	0.021056379	96:78:c6:15:c7:0e	Broadcast	ARP	42	Who has 10.0.0.4? Tell 10.0.0.2
11	0.024597260	96:78:c6:15:c7:0e	Broadcast	ARP	42	Who has 10.0.0.4? Tell 10.0.0.2
12	0.024749565	96:78:c6:15:c7:0e	Broadcast	ARP	42	Who has 10.0.0.4? Tell 10.0.0.2
13	0.028371498	96:78:c6:15:c7:0e	Broadcast	ARP	42	Who has 10.0.0.4? Tell 10.0.0.2
14	0.028532088	96:78:c6:15:c7:0e	Broadcast	ARP	42	Who has 10.0.0.4? Tell 10.0.0.2
15	0.033921491	96:78:c6:15:c7:0e	Broadcast	ARP	42	Who has 10.0.0.4? Tell 10.0.0.2
16	0.033967577	96:78:c6:15:c7:0e	Broadcast	ARP	42	Who has 10.0.0.4? Tell 10.0.0.2
17	0.036817358	96:78:c6:15:c7:0e	Broadcast	ARP	42	Who has 10.0.0.4? Tell 10.0.0.2
18	0.037232614	96:78:c6:15:c7:0e	Broadcast	ARP	42	Who has 10.0.0.4? Tell 10.0.0.2
19	0.043708325	96:78:c6:15:c7:0e	Broadcast	ARP	42	Who has 10.0.0.4? Tell 10.0.0.2
20	0.043921853	96:78:c6:15:c7:0e	Broadcast	ARP	42	Who has 10.0.0.4? Tell 10.0.0.2

这实际上是 ARP 广播数据包在环状拓扑中洪泛导致的，传统网络利用生成树协议解决这一问题。在 SDN 中，不必局限于生成树协议，可以通过多种新的策略解决这一问题。以下给出一种解决思路，请在自学习交换机的基础上完善代码，解决问题：

当序号为 `dpid` 的交换机从 `in_port` 第一次收到某个 `src_mac` 主机发出, 询问 `dst_ip` 的广播 ARP Request 数据包时, 控制器记录一个映射 `(dpid, src_mac, dst_ip)->in_port`。下一次该交换机收到同一 `(src_mac, dst_ip)` 但 `in_port` 不同的 ARP Request 数据包时直接丢弃, 否则洪泛。

代码框架

```
from os_ken.base import app_manager
from os_ken.controller import ofp_event
from os_ken.controller.handler import MAIN_DISPATCHER, CONFIG_DISPATCHER
from os_ken.controller.handler import set_ev_cls
from os_ken.ofproto import ofproto_v1_3
from os_ken.lib.packet import packet
from os_ken.lib.packet import ethernet
from os_ken.lib.packet import arp
from os_ken.lib.packet import ether_types

ETHERNET = ethernet.ethernet.__name__
ETHERNET_MULTICAST = "ff:ff:ff:ff:ff:ff"
ARP = arp.arp.__name__

class Switch_Dict(app_manager.OSKenApp):
    OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]

    def __init__(self, *args, **kwargs):
        super(Switch_Dict, self).__init__(*args, **kwargs)
        self.sw = {} #(dpid, src_mac, dst_ip)=>in_port, you may use it in
mission 2
        # maybe you need a global data structure to save the mapping
        # just data structure in mission 1

    def add_flow(self, datapath, priority, match, actions, idle_timeout=0,
hard_timeout=0):
        dp = datapath
        ofp = dp.ofproto
        parser = dp.ofproto_parser
        inst = [parser.OFPInstructionActions(ofp.OFPIT_APPLY_ACTIONS,
actions)]
        mod = parser.OFPFlowMod(datapath=dp, priority=priority,
                                idle_timeout=idle_timeout,
                                hard_timeout=hard_timeout,
                                match=match, instructions=inst)

        dp.send_msg(mod)

    @set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
    def switch_features_handler(self, ev):
        msg = ev.msg
```

```

        dp = msg.datapath
        ofp = dp.ofproto
        parser = dp.ofproto_parser
        match = parser.OFPMatch()
        actions = [parser.OFPActionOutput(ofp.OFPP_CONTROLLER,
ofp.OFPCML_NO_BUFFER)]
        self.add_flow(dp, 0, match, actions)

@set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
def packet_in_handler(self, ev):
    msg = ev.msg
    dp = msg.datapath
    ofp = dp.ofproto
    parser = dp.ofproto_parser

    # the identity of switch
    dpid = dp.id
    # the port that receive the packet
    in_port = msg.match['in_port']
    pkt = packet.Packet(msg.data)
    eth_pkt = pkt.get_protocol(ethernet.ethernet)
    if eth_pkt.ethertype == ether_types.ETH_TYPE_LLDP:
        return
    if eth_pkt.ethertype == ether_types.ETH_TYPE_IPV6:
        return
    # get the mac
    dst = eth_pkt.dst
    src = eth_pkt.src
    # get protocols
    header_list = dict((p.protocol_name, p) for p in pkt.protocols if
type(p) != str)
    if dst == ETHERNET_MULTICAST and ARP in header_list:
        pass
    # you need to code here to avoid broadcast loop to finish mission 2

    # self-learning
    # you need to code here to avoid the direct flooding
    # having fun
    # :)
    # just code in mission 1

```

结果示例

解决 ARP 数据包在环状拓扑中的洪泛问题后，UCLA 和 UTAH 之间可以 ping 通，并且流表项的匹配次数明显减少：

```

mininet> dpctl dump-flows
*** s1 -----
cookie=0x0, duration=4.268s, table=0, n_packets=3, n_bytes=294, hard_timeout=5, priority=1,in_port="s1-eth2",dl_dst=ba:76:7d:af:5d:96 actions=output:"s1-eth4"
cookie=0x0, duration=4.266s, table=0, n_packets=3, n_bytes=294, hard_timeout=5, priority=1,in_port="s1-eth4",dl_dst=2e:25:1f:f6:ac:83 actions=output:"s1-eth2"
cookie=0x0, duration=33.914s, table=6, n_packets=33, n_bytes=3032, priority=0 actions=CONTROLLER:65535
*** s2 -----
cookie=0x0, duration=4.275s, table=0, n_packets=3, n_bytes=294, hard_timeout=5, priority=1,in_port="s2-eth1",dl_dst=ba:76:7d:af:5d:96 actions=output:"s2-eth2"
cookie=0x0, duration=4.269s, table=0, n_packets=3, n_bytes=294, hard_timeout=5, priority=1,in_port="s2-eth2",dl_dst=2e:25:1f:f6:ac:83 actions=output:"s2-eth1"
cookie=0x0, duration=33.917s, table=6, n_packets=16, n_bytes=1314, priority=0 actions=CONTROLLER:65535
*** s3 -----
cookie=0x0, duration=33.920s, table=6, n_packets=21, n_bytes=1970, priority=0 actions=CONTROLLER:65535
*** s4 -----
cookie=0x0, duration=4.276s, table=0, n_packets=3, n_bytes=294, hard_timeout=5, priority=1,in_port="s4-eth2",dl_dst=ba:76:7d:af:5d:96 actions=output:"s4-eth1"
cookie=0x0, duration=4.275s, table=0, n_packets=3, n_bytes=294, hard_timeout=5, priority=1,in_port="s4-eth1",dl_dst=2e:25:1f:f6:ac:83 actions=output:"s4-eth2"
cookie=0x0, duration=33.922s, table=6, n_packets=24, n_bytes=2152, priority=0 actions=CONTROLLER:65535

```

3. 附加题

实验任务二只给出了一种参考方案，SDN 中还有多种方案可供选择，请尝试设计实现一种新的策略解决环路广播问题。

附加题占分值较小，主要面向学有余力的同学，请结合自身实际决定是否选做。

4. 拓展资料

- SDN论坛: [sdnlab](http://sdnlab.com)
- 关于Mininet的更多资料: [Mininet Doc](#), [Mininet API](#)
- 关于OS-Ken App开发的更多资料: [OS-Ken Documentation Ryu Book](#)